

Task 控制台设计审查文档

实验侧可配置仪表盘 / frontend 组件 / 扩展到真实设备

三层解耦、SignalHub/Feed/TaskConsole、面板几何与密封 API、Panel
Editor、真机接线

2026-06-12

Contents

1	这份文档讲什么	1
1.1	覆盖范围	1
1.2	涉及的源文件	1
1.3	如何重建本文档	2
2	总览: 实验侧的可配置仪表盘	3
2.1	它解决什么问题	3
2.2	设计原则 (贯穿全文)	3
3	三层架构与解耦	5
3.1	数据流	5
3.2	为什么这样分	5
4	SignalHub: 数据契约	6
4.1	发布侧	6
4.2	消费侧	6
4.3	线程安全与 version 跳刷	6
4.4	约定俗成的信号名	6
5	Feed: 数据生产者	8
5.1	ExperimentFeed 基类	8
5.2	VirtualLoadingFeed: 自含虚拟装载实验	8
5.3	多 feed 做 A-B	8
6	面板模型	9
6.1	五种面板类型 (kind)	9
6.2	来源 = 表达式	9
6.3	错误隔离与形状自适应	9
7	面板几何:confocal 模数规则	11
7.1	所有 kind 共享同一块 plot 区域	11
7.2	显示缩放:qt_canvas 三不变量	11

8	卡片与布局	12
8.1	卡片 = 带标题的 Fluent 卡	12
8.2	模数闭合: footer 吸收差额	12
8.3	拖动吸附 + 推挤防重叠	12
9	Setting 弹窗: 轻量配置	13
9.1	声明式参数: 加一个旋钮 = 一行	14
10	Panel Editor 标签页: 重控件之家	15
10.1	为什么是一个标签页	15
10.2	快照模型 (关键设计)	16
10.3	编辑器控件 (详细参考 confocal)	16
11	暂停语义: 两个独立的冻结	17
12	命名任务仪表盘	18
12.1	状态序列化	18
13	性能	19
13.1	瓶颈定位	19
13.2	优化 (全部 logic/appearance-neutral)	19
14	frontend 密封 API 契约	20
15	frontend 绘图组件参考	21
15.1	工厂与 live 类	21
15.2	LiveSiteMap (本轮新增)	21
15.3	DataFigure (拟合/存图/单位)	21
15.4	新增一种 plot type 的方法 (以 sites 为模板)	21
16	扩展到真实设备	23
16.1	最小接线 (notebook, 自管循环)	23
16.2	后台 feed 类 (推荐的真机封装)	24
16.3	对接 references 里的 Rb87 qCMOS 读出	24
16.4	真机 vs 虚拟的唯一差别	25
17	测试与验收	26
18	公共接口与内部结构索引	27
18.1	对外公共 (notebook/实验室会调)	27
18.2	task_console.py 内部要点	27
19	已知限制与待决方向	28
19.1	我的建议	29

1

这份文档讲什么

这是一份**独立的设计审查文档**，专讲实验侧的 Task 控制台 (task console) 及其依赖的 frontend 组件。目标是把**设计思路、实现方式、以及如何接到真实设备上**一次讲透，供你审查并决定下一步方向。它不是面向终端用户的操作手册（那部分在 frontend 中文手册的“Task 控制台”与“对外接口完整参考”两章），而是面向**维护者与你本人**的架构说明。

1.1 覆盖范围

- 动机：为什么要做实验侧仪表盘，它在整个系统里的位置。
- 三层架构与解耦：SignalHub / Feed / TaskConsole，以及它们**只通过信号集线器耦合**这一原则。
- 每一层的契约与实现细节，连同关键的“坑”与设计取舍。
- 面板几何的 confocal 模数规则，以及 frontend 的**密封 API 契约** (美术/几何/dpi 由 frontend 拥有)。
- Setting 弹窗、Panel Editor 标签页、暂停语义、命名任务、状态保存、性能优化。
- **如何扩展到真实设备** (重点章节)，含与 references 里 Rb87 qCMOS 读出流程的对接。
- 测试覆盖、公共/内部函数索引、已知限制与待决方向。

1.2 涉及的源文件

frontend/task_console.py 控制台主体：TaskConsole、PanelCard、PanelConfig、TaskConsoleState、Panel Editor、命名任务。

neutral_atom/core/signals.py SignalHub：线程安全命名信号环。

neutral_atom/operations/feeds.py ExperimentFeed / VirtualLoadingFeed：数据生产者循环。

frontend/live.py 绘图工厂 plot/panel_plot、五个 live 绘图类、FigureSpec 几何、panel_plot_spec。

frontend/qt_canvas.py EmbeddedFigureCanvas/panel_canvas: 高 DPI 嵌入封装。

frontend/qt_fluent.py Fluent 控件库 + resolve_fluent_auto_scale + FluentTabWidget。

frontend/data_figure.py DataFigure: 拟合 / 存图 / 单位换算 (编辑器复用)。

task_console.py / task_console.bat 根启动器。

1.3 如何重建本文档

源是同目录的 task_console_design_zh.texbody; 运行 docs/task_console_design/build.py 会重新生成 assets/ 里的截图并在临时目录里编译, **只把 PDF 拷回本目录** (走 frontend 的密封 PDF 接口 render_notes_pdf, 不留任何.tex/.aux/.log 等中间产物)。

2

总览: 实验侧的可配置仪表盘

2.1 它解决什么问题

脉冲编辑器 (pulse GUI) 解决的是**实验的输出侧**——怎么定义并下发时序到 FPGA。Task 控制台解决的是**实验的输入/监控侧**——实验跑起来之后, 怎么**实时看**原子装载、计数分布、读出保真度、逐站占据这些量, 并且让这套监控**可配置、可保存、可在不同实验间复用**。

一句话: 它是一个网格仪表盘, 里面摆任意多个**实时面板**, 每个面板用一小段**表达式**接到实验信号上, 整套布局存成一个命名的“任务”, 需要时一键载回。

2.2 设计原则 (贯穿全文)

1. **三层解耦, 只经 hub 通信**。采集线程、数据生产者、GUI 三者互不直接引用, 唯一的共享态是 `SignalHub`。这让虚拟实验与真机、单 feed 与多 feed、notebook 与独立窗口都能复用同一套面板。
2. **数据来源是表达式, 不是下拉框**。面板不绑定到某个固定信号, 而是跑一行 Python (`value = ...`), 所以“两个实验装载率相减”这种需求天然支持, 且新增信号无需改 GUI。
3. **美术/几何由 frontend 拥有, 对外密封**。面板尺寸只能从有限预设里选, dpi/边距/字号/阴影/缩放规则外部都改不了——这样任意类型/尺寸的面板能严丝合缝拼接, 且在不同 Windows DPI 下不崩。
4. **轻控件常驻、重控件进标签页** (借鉴 confocal)。Setting 弹窗只放高频的数据/显示配置; 命令行拟合、x/y 范围、单位换算、存图这些重控件放到 Panel Editor 标签页, 避免 Setting 过载。
5. **虚拟与真机零修改通用**。虚拟 feed 发布的信号名 (`frame/counts/occupied/centers/rate`) 就是约定; 真机沿用同名发布, 同一个命名任务在两边都能用。

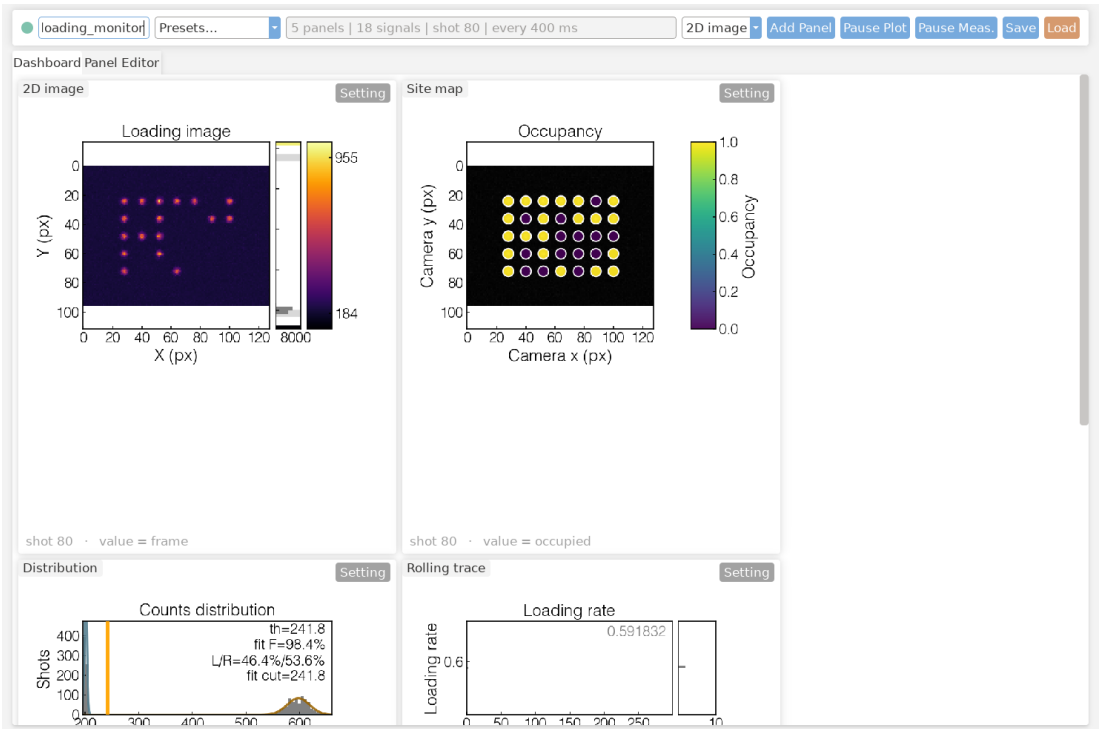
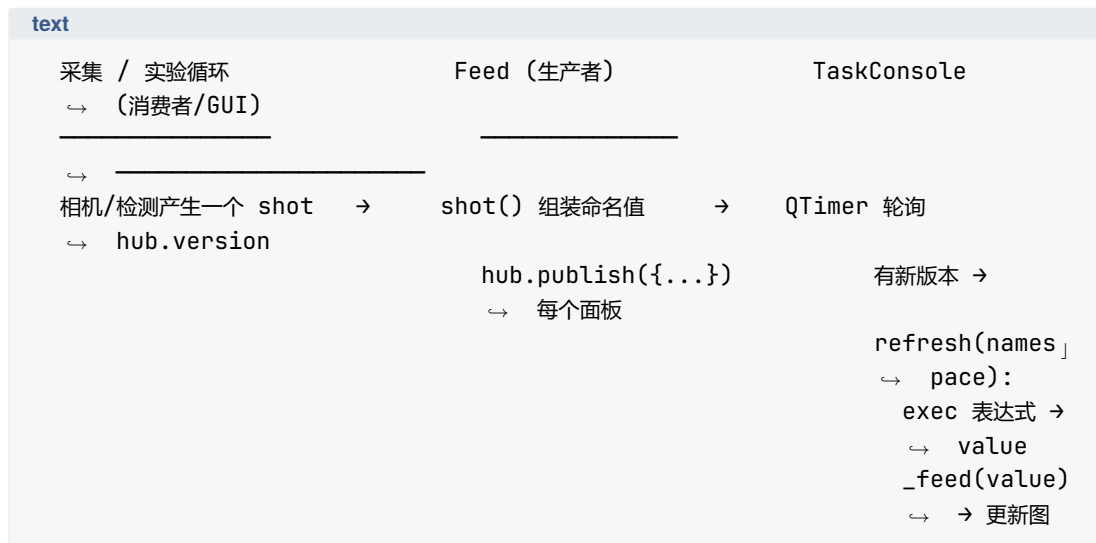


Figure 2.1: 默认任务 `atom\_loading\_monitor`: 左上相机帧、右上占据站点图、计数分布 (自动双高斯)、装载率滚动轨迹、底部逐站装载率。顶栏: 任务名 / Presets 下拉 / 摘要 / 面板类型 / Add Panel / Pause Plot / Pause Meas. / Save / Load; 两个标签页 Dashboard 与 Panel Editor。

3

三层架构与解耦

3.1 数据流



三者只通过 **SignalHub** 耦合：

- Feed 只知道“往 hub publish 命名值”，不知道有没有 GUI、有几个面板。
- TaskConsole 只知道“从 hub 读最新值/历史”，不知道数据是虚拟渲染的还是真相机来的。
- 采集循环（真机）可以是 Feed 的后台线程，也可以是 notebook 里你自己的 **for** 循环——只要每个 shot 调一次 **hub.publish**。

3.2 为什么这样分

这套分层直接决定了“扩展到真机”有多容易（见第 16 章）：把虚拟 Feed 换成真机采集循环，GUI 一行都不用改。解耦的代价是多了一层 hub 的拷贝/版本管理，但换来的是**可测试**（同步 **step()** 驱动，无线程）、**可移植**（布局 JSON 不含任何机器相关的几何）、**可复用**（多 feed 同 hub 做 A—B）。

4

SignalHub: 数据契约

SignalHub (`neutral\_atom/core/signals.py`) 是采集侧与 GUI 之间**唯一的共享态**: 一个线程安全的命名信号环。

4.1 发布侧

`hub.publish(values: Mapping[str, object], *, shot=None) → int`: 把一批命名值写入。每个值被**强制转成 float ndarray (标量变成 0 维) 并拷贝**——所以生产者可以随意复用自己的缓冲区, 不必担心 GUI 读到半写状态。返回新的 version 号。

4.2 消费侧

`latest(name)` 最新值: 0 维返回 float, 否则返回 ndarray 的拷贝; 不存在抛 `KeyError` (错误信息列出可用名)。

`history(name, n=None) → ndarray` 最近若干 shot 堆叠, **形状 (shots, *value_shape)**, axis 0 是 shot; 不足 n 返回现有; 值的形状变过时只保留最近一段同形状连续区。

`names() → list[str]` 当前所有信号名 (排序)。Setting 的信号下拉就是调它。

`snapshot\_latest() → dict` 一致快照 (一次性拿到所有最新值) ——这就是面板表达式求值的命名空间底座。

version / shot 版本计数 / shot 计数。控制台用 version 跳过“没有新数据”的刷新。

4.3 线程安全与 version 跳刷

publish 与读取都在锁内做副本, 所以采集线程和 GUI 线程互不撕裂。控制台的 `\_tick` 先比较 `hub.version`: 没变就直接返回, 不做任何绘图工作——空闲时近乎零开销。

4.4 约定俗成的信号名

虚拟 feed 发布、且真机建议沿用的一组名字 (沿用它们 = 同一命名任务在虚拟/真机间零修改通用):

信号	形状	含义
frame	(H, W)	最新相机帧
counts	(N_sites,)	逐站 ROI 计数
occupied	(N_sites,)	逐站 0/1 占据
rate	标量	滚动装载率
rate_sites	(N_sites,)	逐站滚动装载率
rate_grid	(rows, cols)	网格形状的逐站率
centers	(N_sites, 2)	标定的站点中心 (相机像素)
thresholds	(N_sites,)	逐站阈值
shot	标量	shot 计数

5

Feed: 数据生产者

5.1 ExperimentFeed 基类

`neutral\_atom/operations/feeds.py` 的 `ExperimentFeed` 拥有发布循环；子类只实现一个 `shot()` (产一个 shot 的 {名: 值})。

step() 同步跑一个 shot 并 publish (加 `prefix`)。测试/notebook 用——无线程、确定性。

start(rate_hz=5.0) 守护线程按速率循环 `step`；记住 `rate\_hz` 以便 Pause/Resume 用原速率恢复。线程里 `shot()` 抛异常会退避重试而不是静默杀掉线程（控制台显示陈旧数据）。

stop() / running 停线程 / 是否在跑。

prefix 多 feed 同 hub 时的命名前缀（如 `b\_`）——这就是 A-B 的基础。

5.2 VirtualLoadingFeed: 自含虚拟装载实验

`VirtualLoadingFeed` 内部用 `VirtualTrapArray` 渲染荧光帧，并用与真实流水线相同的 `find\_site\_centers / roi\_counts / Otsu` 阈值自标定。每个 shot 发布上表那组信号。这一点很关键：虚拟 feed 不是“假数据”，它走的是和真机一样的检测原语，所以面板表达式、拟合、站点图在虚拟上验证通过 = 在真机上同样成立。

5.3 多 feed 做 A-B

默认启动开两个 feed：主 feed（无前缀）+ 一个 `b\_` 前缀的副 feed（装载率不同）。于是 `value = rate\_grid - b\_rate\_grid` 这种跨实验相减开箱即用。

6

面板模型

6.1 五种面板类型 (kind)

kind	绘图类	用途
2d	Live2DDis	图像 + 侧分布 + colorbar + 可拖 clim (相机帧、网 格率)
sites	LiveSiteMap	原子阵列站点图: 每站一个实心圆按值着色 + 可垫相 机帧
1d	Live1D	一维折线 (逐站量、横轴站号)
monitor	LiveLiveDis	标量滚动轨迹 + 侧分布
hist	HistogramFigure	直方图 + 自动双高斯 + 可拖阈值线 + 保真度

6.2 来源 = 表达式

每个面板的数据来源是一行 Python, 赋给 `value` (与脉冲 GUI 的 Scan 页同一套“本地可信代码”信任模型——只跑你自己写的):

Python

```
value = frame                # 最新相机帧 -> 2D
value = occupied             # 每站 0/1 占据 -> Site map
value = rate_sites           # 逐站装载率 -> Site map / 1D
value = history('counts', 200).ravel() # 最近 200 shot 计数 -> Distribution
value = rate_grid - b_rate_grid # 两个实验相减 -> 2D
```

命名空间 = hub 的一致快照 (每个信号名直接可用) + `history/latest/names/shot/np/math`。

6.3 错误隔离与形状自适应

`PanelCard.refresh(namespace)` 把表达式求值与喂图包在 try 里: **某个面板表达式报错, 错误只显示在该面板自己的状态行 (红字)**, 其余面板照常刷新, 控制台循环不受影响。`\_coerce` 按 kind 校验/降采样 value (2D 对 >192 维做步进降采样保证每 tick 便宜; sites 限站点数)。`\_feed` 里只有当

value 形状变化（换表达式/尺寸/参数）才重建该面板的图，否则增量更新。

7

面板几何:confocal 模数规则

这是面板能整齐拼接的根本，也是 frontend 密封 API 的核心体现。

7.1 所有 kind 共享同一块 plot 区域

`panel\plot\_spec(size)` (`frontend/live.py`) 对**每一种 kind** 返回同一套几何：

- `PANEL\_SIZES = ("1x2", "2x2", "1x4", "2x4")`，含义是“行 × 列的半单元数”。
- `2x2` = frontend 的标准 plot 区域，数据区 480×360 ，边距 `(110, 110, 100, 70)` (左右下上；顶 70 是 title 槽，常备，与有没有标题无关)。
- `1x2` = 半高，`2x4` = 双宽。
- 2D 与 Site map 在**自己的区域内部**做 `[0.75, 0.1, 0.1]` 水平分割 (主图 + 侧分布 + colorbar)，所以 `2x2` 主图恰好 360×360 **正方形**且与 colorbar 顶底对齐——这正是不同类型面板能整齐排版的原因。

对外接口只有 `panel\plot(data, kind=, size=)` 与 `panel\plot\_spec(size)`：**尺寸是唯一旋钮**，几何/dpi/边距不可外部配置 (`panel\_size\_cells` 校验尺寸，非预设抛错)。

7.2 显示缩放:qt_canvas 三不变量

面板图通过 `qt\_canvas.panel\_canvas(figure)` 嵌入，封装常数 `PANEL\_DISPLAY\_SCALE=0.7`。`EmbeddedFigureCanvas` 维持三条不变量 (在 `QT_SCALE_FACTOR` 1.0/1.25/1.5 三档离屏复现 + 验证)：□ `figure.inches` 构造后永不变；□ `fig.dpi` = 设计 dpi × 真实屏幕比例 (标准 retina，显示因子绝不进 dpi)；□ 控件逻辑尺寸 = 设计 px × `display_scale`。`resizeEvent` 故意不从控件尺寸反推 figure——这是高 DPI 真机布局曾经崩坏的根本修复 (见性能/契约章)。

8

卡片与布局

8.1 卡片 = 带标题的 Fluent 卡

`PanelCard` 继承 `FluentGroupBox`, 卡框标题 = 面板类型 (2D image / Site map / ...), 带 fluent 阴影 (`CachedDropShadow`, 按 silhouette 缓存、move 时不重新模糊)。画布下方、卡片底部一行灰色状态行 (当前 shot + 来源表达式回显)。标题行右端一个 `Setting` 文字按钮 (出错变红、错误进 tooltip)。

8.2 模数闭合:footer 吸收差额

图本身 = plot 区域 + 边距, 边距不随数据区折半, 所以**光靠 canvas 永远对不齐网格**。卡框的状态行区按行数**吸收这点差额**: `\_card\_size(size) = rows$\times$ $pitch\_y - gap`, 使得**一张 2x2 卡严格等于两张 1x2 卡叠起来加一道间隙** (竖直 pitch 323、水平 259, 契约测试锁定 $2 \times \$315 + 8 = 638$)。任意尺寸混排严丝合缝。

8.3 拖动吸附 + 推挤防重叠

排版不用填行列数字: **按住卡片边框 (画布以外的空白) 拖动**, 松手 `\_pos\_to\_slot` 吸附到网格槽并写回 config。卡片间**不允许重叠**——`\_resolve\_collisions` 在落点被占时把被压住的卡向下**级联挤开**, 拖动的卡始终保住目标槽 (推挤只增加 row、不动 active 卡, 保证收敛)。

9

Setting 弹窗: 轻量配置

卡片标题行的 **Setting** 按钮打开该面板的设置弹窗 (confocal 风格), 从上到下三/四组——**换数据动 Source、换显示动 Display、即时生效:**

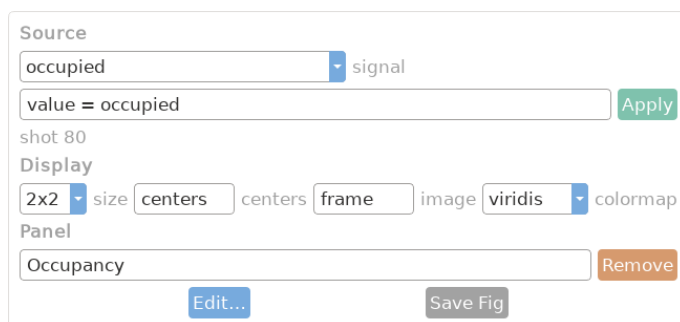


Figure 9.1: Site map 面板的 Setting 弹窗:Source(信号下拉/表达式 +Apply)、Display(size + 声明式参数 centers/image/colormap)、Panel(标题/Edit.../Save Fig/Remove)。边框按 `objectName` 作用域, 不级联到子控件。

Source `signal` 下拉列出 hub 此刻真实存在的所有信号名——选一个面板立即切到 `value = 信号名`; 要做运算就在表达式框写一行点 **Apply**。出错时 **Setting** 按钮变红。

Display `size` 预设 + 该类型自己的**声明式参数** (`PANEL\_PARAMS` 里一行一个 `ParamSpec`, 控件自动生成、随布局 JSON 保存): 2D 的 colormap; Site map 的 centers/image 信号名 + colormap; Rolling/1D 的 relim (tight/normal); Rolling 的历史长度; Distribution 的 bins。

Fit (仅线型面板) 预设模型快速拟合当前数据 (重的命令行拟合在 Editor 里)。

Panel 标题、**Edit...** (跳 Panel Editor)、**Save Fig** (一键存图)、**Remove**。

Note

坑 (已修): 弹窗边框必须按 `objectName` 作用域 `QFrame\#zlcPanelSettings`。裸 `QFrame` `\{ border \}` 会级联到所有 `QFrame` 派生子控件——而 `QLabel/QComboBox/QSpinBox` 内部都是 `QFrame`, 曾导致每个标签/控件周围多出 1px 杂线。容器描边一律用 `\#objectName` 或


```
paintEvent.
```

9.1 声明式参数: 加一个旋钮 = 一行

`ParamSpec(key, label, kind, default, choices=, lo=, hi=, tooltip=)`, `kind` 取 `choice/int/signal`。Setting 控件、JSON 持久化、改动后重建全部自动跟上; `\_set\_param` 是唯一写入口。要给某类面板加一个可配置项, 就在 `PANEL\_PARAMS[kind]` 加一行——不用写 bespoke 控件。

10

Panel Editor 标签页: 重控件之家

10.1 为什么是一个标签页

confocal 里可配置项极多 (命令拟合、x/y 范围、单位、存图.....), 全塞进 Setting 弹窗会过载。借鉴脉冲 GUI 的多标签布局, 控制台是一个 `FluentTabWidget`: `Dashboard` (实时网格) + `Panel Editor` (重控件)。在面板 Setting 里点 `Edit...` 即跳到编辑页。

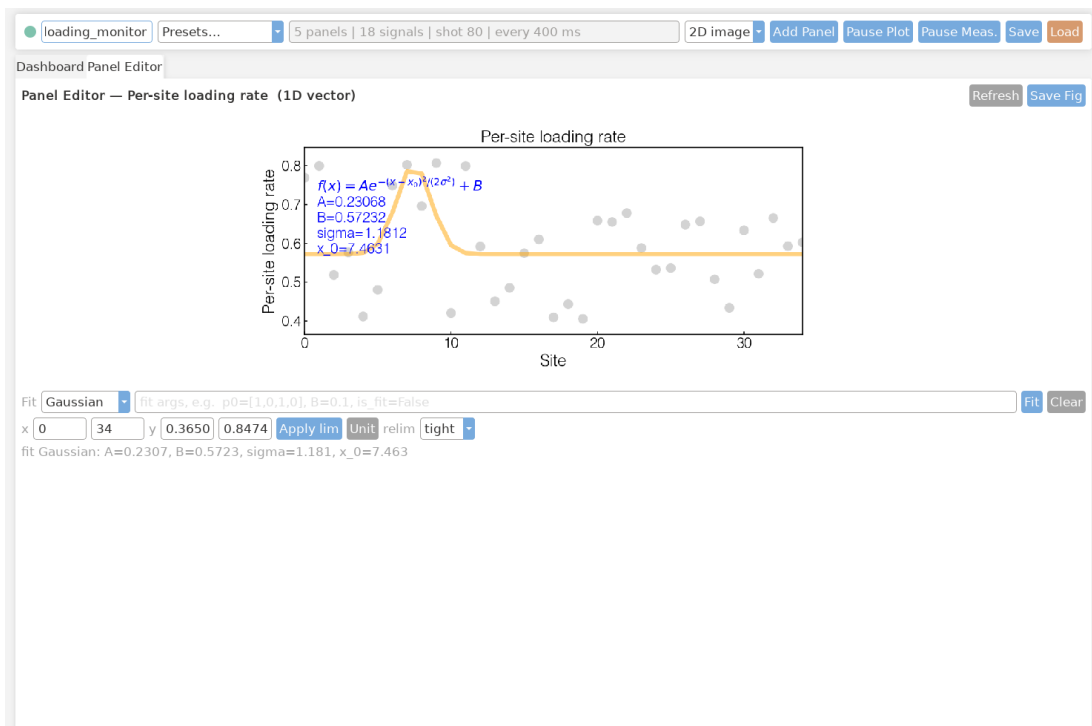


Figure 10.1: Panel Editor: 绑定面板当前数据的快照 (灰散点), 命令行拟合叠加 (橙色高斯曲线 + 蓝色公式 + 参数读数), Fit 行 (模型 + 自由文本 args+Fit+Clear), Axes 行 (x/y limits+Apply+Unit+reim), Save Fig + Refresh, 状态行。

10.2 快照模型 (关键设计)

点 Edit 时, 编辑器**绑定该面板当前数据的一份快照**——用 `panel\_plot` 在更大尺寸 (2x4) 重建一个**独立的 plotter** (不是 live 卡那个)。理由:

- 天然契合“冻结细看”工作流 (配合 Pause Plot 更稳): 编辑器看的是一帧定格数据, 不会被实时刷新搅动。
- **不打扰仪表盘**: 编辑/拟合/改范围都作用在编辑器自己的图上, Dashboard 照常实时刷新。
- **Refresh** 重新抓一份当前快照。

10.3 编辑器控件 (详细参考 confocal)

命令行拟合 模型下拉 (Lorentzian/Gaussian/Exp decay/Rabi) + 一行**自由文本参数**, 注入到拟合调用 (本地可信代码, `eval(f"\_df.\{method\}(\{cmd\})")`), 与 confocal 的 : 框同理): `p0=[...]` 给初值、`名字 = 值` 固定某命名参数、`is\_fit=False` 只按初值叠加。橙色拟合曲线 + 公式 + 参数读数叠加在图上, **Clear** 移除。底层是 `DataFigure` (与 notebook 的 `p.data\_figure.lorent()` 同源)。

x/y 范围 四框 + `Apply lim (DataFigure.xlim/ylim)`; 打开时自动回填当前坐标范围。

Unit 在已定义的单位环上循环 (GHz/nm/MHz 或 ns/us/ms, `DataFigure.change\_unit`)。

relim tight/normal, 即时重建编辑快照并记到该面板的 config。

Save Fig < 标题 >.png + 数据 .npz (时间戳) 到 `tasks/` (`DataFigure.save` 返回 {figure, data})。

Note

坑 (已修): 编辑器改 relim **不能走** `card.\_set\_param`——它内部 `\_reset\_plot` 会把 live 卡的 plotter 置 None, 编辑器随后从 `card.plotter` 读快照就会崩。正确做法: 直接写 `card.config.params` + 标脏 + 只重建编辑器快照。

11

暂停语义: 两个独立的冻结

实验里“暂停”有两层含义，控制台给了**两个独立按钮**：

Pause Plot 停面板刷新 timer。**数据仍在流进 hub**，只是画面冻结——正是“冻结后做拟合/细看”的场景。

Pause Meas. 停/启 feeds（实验数据源本身，不再产新 shot）。无可控 feed（如 notebook 自管循环）时禁用。

两者正交：冻结画面不影响采集，暂停采集不影响 timer。典型流程:Pause Plot → 在 Editor 里 Fit/细看 → Resume Plot。

12

命名任务仪表盘

布局（面板、位置、尺寸、表达式、参数）整包存成一个 JSON，文件名即任务名。三个入口等价：

- GUI：顶栏 Presets... 下拉列出全部可载任务（内置 + tasks/ 里每个 JSON），选中即整套载入；Save 默认存到 tasks/任务名.json。Save* 黄星 = 有未保存改动。
- 命令行：task_console.bat -task atom_loading_monitor。
- Notebook：zf.show_task_console(hub=hub, task="atom_loading_monitor")。

内置出厂任务 BUILTIN_TASKS：atom_loading_monitor（装载图 + 占据站点图 + 计数分布 + 装载率 + 逐站条）与 loading_rate_live（轻量装载率监控）。在 tasks/ 里存同名 JSON 即覆盖内置版本——想改造出厂布局，载入、拖一拖、Save 就行。配一套新任务（如 atom_temp_monitor）：载入最接近的现有任务 → 增删面板、把 Source 指到温度实验发布的信号 → 改任务名 → Save。

12.1 状态序列化

TaskConsoleState(schema/version2):name/interval_ms/panels[];每个 PanelConfig 含 kind/title/row/col/size/source/params, to_dict/from_dict 往返。布局 JSON 不含任何几何像素（几何归 frontend），所以跨机器可移植。

13

性能

13.1 瓶颈定位

对 refresh tick 做 cProfile (离屏、5 面板、96×128 帧 + 35 站 + 增长历史): 约 **90% 的时间在两处** **scipy curve_fit**——直方图双高斯拟合 `HistogramFigure.\_fit\_bimodal` 与滚动轨迹高斯拟合 `LiveLiveDis.\_update\_gauss\_fit`, 都在为 scipy 的有限差分 Jacobian 买单。

13.2 优化 (全部 logic/appearance-neutral)

- **解析 Jacobian**: 给两处 `curve\_fit` 提供解析梯度 (`\_bimodal\_jac/\_gauss\_func\_jac`)。同模型同 p0 同 bounds → 收敛参数完全一致 (实测 popt 相对差 1e-10), 只是去掉了有限差分的目标重算。
- 可复用守卫 `BaseLivePlot.\_set\_colorbar\_ticks:limits` 不变时跳过 colorbar 重标 (occupancy 钉 0.1 时常发生), 标签相同故外观一致。
- `set\_status` 仅在 ok↔error 跳变时重设样式表, 不每 tick 重建同一串。

结果: 稳态 refresh 均值 **14.55 → 9.64 ms** (中位 14.43 → 9.61), 完全不改逻辑/外观。拖拽路径 0.015 ms/op (非瓶颈); 阴影按 silhouette 缓存, move 不重新模糊。不做“降低拟合频率”那类会改刷新节奏 = 改外观的优化。

14

frontend 密封 API 契约

frontend 对外暴露一个**小而密封**的接口: 外部只传数据 + 少量语义选项, 拿回正确的美术/几何/dpi/字体, **绝不能通过任何公共接口搞乱视觉设计**。权威定义在 `frontend/\_\_init\_\_.py` 模块文档 + `frontend/AGENTS.md` + `docs/MAINTAINER\_NOTES.md` §18。六条规则:

1. 几何被拥有、永不外传: `plot()/panel\_plot()` 拒绝 `spec/data\_px/margins\_px/dpi/bad\_color/colors` (内部几何走私有 `plot(\_spec=)`)。
2. 尺寸/cmap 是 curated 且入口校验 (`PANEL\_SIZES/panel\_size\_cells`); pulse 几何常量私有化。
3. 一套排版一个 dpi: `style.DEFAULT\_STYLE` 是只读 `MappingProxyType`。
4. 一条缩放规则: `resolve\_fluent\_auto\_scale(GUI)+panel\_canvas/PANEL\_DISPLAY\_SCALE` (嵌入图); `show\_*` 默认 `scale=None`。
5. 带美术的 fluent 控件保持内部: `qt\_fluent.* / qt\_canvas.*` 不进 `\_\_all\_\_` (阴影/边框/圆角是构造细节)。
6. 加公共参数前先分类: `DATA(labels/title/bins/thresholds/relim/cmap-from-list/channels)` 允许; `ART/GEOMETRY/TYPOGRAPHY` 只能在内部类上。

Task 控制台严格遵守: 面板只通过 `panel\_plot+size` 预设作图, 拟合只经 `DataFigure(card.plotter)`, 改图只改 `config` + 重建——从不戳 figure inches/边距。

15

frontend 绘图组件参考

15.1 工厂与 live 类

`plot(data\_x, data\_y, *, kind, update, labels, display, data\_figure, ...)` 是统一工厂 (数组契约 `data\_x=(N,coord\_dim)/data\_y=(N,channel\_dim)`); `panel\_plot(data, *, kind, size)` 是它的仪表盘特化 (不 `display`、不附 `DataFigure`, 由宿主嵌 `canvas`)。五个 live 类: `Live1D/Live2DDis/LiveLiveDis/HistogramFigure/LiveSiteMap`, 都公共导出、可直接构造, 但首选 `plot()`。

15.2 LiveSiteMap(本轮新增)

原子阵列站点图: `data\_x` 是 $(N, 2)$ 站点中心 (相机像素), `data\_y` 是逐站值; 每站一个 `EllipseCollection` 实心圆按值着色, 可选 `image` 垫相机帧底图、`cmap`、`roi\_radius`。同 2D 的 `[0.75, 0.1, 0.1]` 分割, `square` 主图与 `colorbar` 逐像素对齐——融入既有几何而非另起炉灶。

15.3 DataFigure(拟合/存图/单位)

`DataFigure(live\_plot)` 从一个 live 绘图对象建数据句柄。关键方法: `lorent/gaussian/decay/rabi/lorent\_zeeman/...` 接 `p0=/is\_fit=/*kwargs` 固定命名参数)、`xlim/ylim`、`change\_unit`(单位环)、`clear`、`save(path)`(写 `png+npz`、时间戳、返回 `{figure, data}`)。编辑器与 Setting 的拟合/存图全部复用它。

15.4 新增一种 plot type 的方法 (以 sites 为模板)

1. 在 `live.py` 写一个 `BaseLivePlot` 子类, 几何必须走 `panel\_plot\_spec` 的共享区域 (用 `split\_axes\_horizontally` 等), 并进 `plot()` 工厂的 `kind` 分发 + `\_normalize\_kind` 别名。
2. 在 `task\_console.py` 的 `PANEL\_KINDS/\_DEFAULT\_SOURCES/PANEL\_PARAMS` 各加一项; `\_coerce/\_build\_plot/\_feed` 各加一个分支。
3. 若需要辅助信号 (如 `sites` 的 `centers/image`), 在 `\_xxx\_aux(namespace)` 里取并校验, 错误进该面板状态行。

-
4. 可视化改动必须用 `devtools.capture\_user\_view` 三档缩放截图验收 (`parity` 目标对比两 GUI 控件尺寸)。

16

扩展到真实设备

这是你最关心的部分: 虚拟仪表盘怎么变成真机监控台。核心结论: GUI 一行不用改, 只需把虚拟 feed 换成真机采集循环、按约定信号名 publish。

16.1 最小接线 (notebook, 自管循环)

Python

```
from Zou_lab_control import frontend as zf
from Zou_lab_control.neutral_atom.core.signals import SignalHub
import Zou_lab_control.neutral_atom as na

hub = SignalHub()
console = zf.show_task_console(hub=hub, task="atom_loading_monitor")

exp = na.connect("virtual")          # 真机换成对应 config
centers = exp.readout.sitemap(frames=12,
    ↪ display=False).calibration.centers
rate = None
for shot in range(100000):
    frame = exp.camera.acquire(1, sequence=exp.sequence,
                                sequencer=exp.devices.sequencer)[0]
    det = exp.readout.from_image(frame, display=False) # AtomDetection
    occ = det.occupied.astype(float)
    rate = occ.mean() if rate is None else 0.95*rate + 0.05*occ.mean()
    hub.publish({"frame": frame, "counts": det.counts, "occupied": occ,
                "centers": centers, "rate": rate, "shot": float(shot)})
```

要点: □ 沿用约定信号名, `atom\_loading\_monitor` 任务零修改即工作; □ `centers` 来自一次 `sitemap` 标定 (`TrapCalibration.centers`), 让 Site map 面板有站点坐标; □ `readout.from_image` 走的就是虚拟 feed 自标定用的同一套检测原语, 所以行为一致。

16.2 后台 feed 类 (推荐的真机封装)

更干净的做法是写一个 `ExperimentFeed` 子类,把上面循环体放进 `shot()`,然后 `feed.start(rate\_hz=...)` 跑后台线程、`show\_task\_console(hub=hub, feeds=[feed])`。这样 Pause Meas. 按钮能直接停/启采集。骨架:

Python

```
class AtomLoadingFeed(ExperimentFeed):
    def __init__(self, hub, session, *, prefix=""):
        super().__init__(hub, prefix=prefix)
        self.exp = session
        self.centers = session.readout.sitemap(frames=12,
        ↪ display=False).calibration.centers
        self._rate = None
    def shot(self):
        frame = self.exp.camera.acquire(1, sequence=self.exp.sequence,
        ↪ sequencer=self.exp.devices.sequen
        ↪ cer)[0]
        det = self.exp.readout.from_image(frame, display=False)
        occ = det.occupied.astype(float)
        self._rate = occ.mean() if self._rate is None else 0.95*self._rate
        ↪ + 0.05*occ.mean()
        return {"frame": frame, "counts": det.counts, "occupied": occ,
        ↪ "centers": self.centers, "rate": self._rate}
```

线程安全:`shot()` 只产数据、`publish` 负责拷贝,GUI 在另一线程读副本——生产者可放心复用缓冲。阻塞式硬件采集放在 `feed` 线程,绝不放 UI 线程。

16.3 对接 references 里的 Rb87 qCMOS 读出

`references/.../rb87\_readout\_v16` 是一套真实 qCMOS 读出流程:4-shot group(20ms 参考 $\times 3$ + 短曝光读出)、模板找站点 (`find\_sites\_from\_template`, 35 站)、PSF 加权提取标量信号 (`box/psf/weighted-logpsf`)、20ms 参考双高斯定阈值、短曝光 per-site 阈值训练出保真度 `F\_bal`、单帧 `single\_predict` 出占据。映射到本仪表盘:

Rb87 读出产出	仪表盘信号 / 面板
短曝光帧	<code>frame</code> → 2D image
<code>single_predict</code> 占据 (35,)	<code>occupied</code> → Site map(占据)/1D
per-site 信号 (photons)	<code>counts</code> → Distribution(双高斯 + 阈值)
站点格点中心 (35,2)	<code>centers</code> → Site map 站点坐标
装载率 = mean(occupied)	<code>rate</code> → Rolling trace
逐站 <code>F_bal</code> / 阈值	<code>fidelity_sites / thresholds</code> → 1D / Site map

也就是说: 把 Rb87 读出循环的每个 group 算完后 `hub.publish` 这些量,现成的 `atom\_loading\_monitor` 任务就直接显示; 保真度、参考标签有效率这类慢量另开 Rolling trace/1D 面板把 Source 指过去即可。`rb\_plots.py` 里现有的画图 (per-site 分布网格、参考双高斯、ablation 曲线) 中, 分布/站点图/滚动物量都已被现有面板覆盖:时序漂移、per-site 保真度条、ablation 曲线是目前面板覆盖不到、值得新增的类型 (见下章待决方向)。

16.4 真机 vs 虚拟的唯一差别

虚拟 feed 用 `VirtualTrapArray` 渲染, 真机用相机; 两者都走 `find\_site\_centers/roi\_counts/Otsu`。
所以: 在虚拟上设计好的任务、验证过的面板/拟合, 接真机时不需要改任何 GUI 代码, 只换数据源。这正是三层解耦的回报。

17

测试与验收

- 契约/渲染:hub 契约、feed 信号 + 收敛、五种面板渲染 + 更新、A-B 表达式 + 错误隔离、布局 JSON 往返、增删 + 脏星 + 存盘、命名任务往返 + 覆盖、信号下拉、Site map 坏 centers 隔离。
- 几何/缩放:confocal 模数契约 (签名 =["size"]、dpi、margins、PANEL_SIZES)、卡片 tiling($2 \times (1 \times 2) + \text{gap} = 2 \times 2$)、EmbeddedFigureCanvas 三档缩放不变量、两 GUI auto-scale parity。
- 密封 API:plot/panel_plot 拒绝几何 kwargs、DEFAULT_STYLE 只读、pulse_plot_spec 无 margins/dpi。
- 性能中立: 解析 Jacobian 与有限差分收敛参数 $1e-10$ 一致。
- 编辑器: 多 tab、Edit 绑定快照、命令拟合叠加、Apply lim、Save Fig 产 png+npz、Clear。
- PDF 接口:render_tex_pdf 临时目录只产 pdf、render_notes_pdf docs 只留 pdf。
- **可视化验收强制规则:** 任何 UI/plot 改动必须 capture_user_view 三档 QT_SCALE_FACTOR(1.0/1.25/1.5) 整窗截图 + 1:1 裁剪;DPR=1 离屏通过不算通过。

当前全量 286 测试绿。

18

公共接口与内部结构索引

18.1 对外公共 (notebook/实验室会调)

`zf.show\_task\_console(*, hub, state=None, task=None, feeds=(), scale=None, window\_ratio=0.8)`
开控制台窗口。

`zf.panel\_plot(data\_x, data\_y=None, *, kind, size="2x2", \*\*kwargs)`
仪表盘面板工厂。

`zf.plot(...)` 通用绘图工厂 (notebook)。

`SignalHub / publish/latest/history/names/snapshot\_latest` 数据契约。

`ExperimentFeed/VirtualLoadingFeed` 生产者基类与虚拟实现。

18.2 task_console.py 内部要点

`PANEL\_KINDS/PANEL\_PARAMS/ParamSpec/FIT\_MODELS/FITTABLE\_KINDS;BUILTIN\_TASKS/resolve\_task\`
几何 `\_grid\_pitch/\_card\_size/\_pos\_to\_slot/\_resolve\_collisions;PanelCard(\_build\_setting`

19

已知限制与待决方向

下面是**留给你决定**的方向（按我判断的价值排序），每条注明现状与代价。

1. **真机 feed 类入库**。现在只有 `VirtualLoadingFeed`；建议加一个从 `NeutralAtomSession` 采集循环 publish 的 `AtomLoadingFeed`（上一章给了骨架），让真机开箱即用。代价小、价值高。
2. **保真度/温度等“派生量”的标准产出**。Rb87 读出的 `F_bal`、参考标签有效率、release-recapture 温度等慢量，目前要用户自己在循环里算并 publish。可考虑在 readout 子系统里提供“逐 shot 派生量”helper，使 `atom\_temp\_monitor/fidelity\_monitor` 这类任务也开箱即用。
3. **时序/漂移面板**。现有 monitor 是单标量滚动；Rb87 的“站点中心随 group 漂移”“逐站 `F_bal` 条形”“drop-worst-sites ablation 曲线”是新面板类型（多线/条形/scatter）。要不要加，取决于你日常盯哪些量。
4. **编辑器是否要直接编辑 live 图**。目前编辑器是**快照**（冻结、独立）。若你想在编辑器里直接调 live 卡的 `clim`/范围并实时反映到 Dashboard，需要 reparent live canvas 或双向同步，复杂度上一个台阶。我倾向保持快照模型（简单、安全），除非你明确要 live 编辑。
5. **2D 的 `clim` 数值框**。现在 2D `clim` 靠侧分布拖拽（交互式）；要不要在编辑器给 2D 加 `clim` 数值框 + `cmap` 下拉（`DataFigure.change\_cmap` 已有）。
6. **拟合模型扩展**。现有 Lorentzian/Gaussian/Exp decay/Rabi；Rb87 常用的双高斯（分布面板已自动做）、`lorentz_zeeman` 等可加进编辑器下拉（`DataFigure` 已支持）。
7. **控制台 ↔ 脉冲 GUI 联动**。同一进程里共享 sequencer，在控制台里直接触发 `on_pulse` / 改 detection time 并看结果——把“配置脉冲 → 采集 → 监控”闭环到一个窗口。这是较大的一步，需要想清交互。
8. **多 group / 4-shot 结构**。Rb87 是 4-shot group；若要在仪表盘层面理解 group（而不是把 group 算完再 publish 一个标量），需要 hub/feed 支持 group 语义。多数情况下“循环里算完再 publish”已足够，不必上 group 抽象。

19.1 我的建议

按价值/代价, 我建议下一步优先 (1) 真机 feed 类 + (2) 派生量 helper, 让真机监控开箱即用;(3) 时序/保真度面板按你实际要盯的量再定。(4)(7) 是更大的架构决定, 建议你审查本文后明确方向再做。